1. Podstawowe struktury warunkowe

- **Prosty `if`:**

  Bash

  ```
  if [[ $VAR -gt 10 ]]; then
      echo "The variable is greater than 10."
  fi
  ```

- **Instrukcja `if-else`:**

  Bash

  ```
  if [[ $VAR -gt 10 ]]; then
      echo "The variable is greater than 10."
  else
      echo "The variable is equal or less than 10."
  fi
  ```

- **Instrukcja wielowariantowa `if-elif-else`:**

  Bash

  ```
  if [[ $VAR -gt 10 ]]; then
      echo "The variable is greater than 10."
  elif [[ $VAR -eq 10 ]]; then
      echo "The variable is equal to 10."
  else
      echo "The variable is less than 10."
  fi
  ```

2. Operatory porównania liczb całkowitych

Wcięte w przykładach operatory numeryczne oznaczają:

- `-eq` (*equal*) – równe (`=`)
- `-ne` (*not equal*) – różne (`!=`)
- `-gt` (*greater than*) – większe niż (`>`)
- `-ge` (*greater than or equal*) – większe lub równe (`>=`)
- `-lt` (*less than*) – mniejsze niż (`<`)
- `-le` (*less than or equal*) – mniejsze lub równe (`<=`)

3. Zagnieżdżanie i operatory logiczne (&&, ||)

- **Zagnieżdżony `if` (instrukcja w instrukcji):** Pozwala sprawdzać kolejne warunki wewnątrz już spełnionego bloku (np. szukanie największej z trzech liczb za pomocą `VAR1`, `VAR2`, `VAR3`).
- **Wiele warunków w jednej linii (&& oraz `||`):**
  - && (AND – i): Wykonuje się, gdy oba warunki są prawdziwe.

    Bash

```
if [[ $VAR1 -ge $VAR2 ]] && [[ $VAR1 -ge $VAR3 ]]; then
    echo "$VAR1 is the largest number."
fi
```

- `||` (OR – lub): Wykonuje się, gdy przynajmniej jeden warunek jest prawdziwy.

  Bash

```
if [[ $USER_INPUT -lt 10 || $USER_INPUT -ge 100 ]]; then
    echo "Podales liczbe która nie ma dwóch znaków"
fi
```

4. Praktyczne przykłady z zadań

- **Sprawdzanie reszty z dzielenia (liczba parzysta / nieparzysta z użyciem modulo `$((x%2))`)**:

  Bash

```
echo -n "Wprowadź liczbę: "
read x
if [ $x == 0 ]; then
    echo "Liczba jest równa 0"
elif [ $((x%2)) == 0 ]; then
    echo "liczba jest parzysta"
else
    echo "liczba jest nieparzysta"
fi
```